



Pashov Audit Group

Reserve Security Review

August 31st 2026 - September 09th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. Executive Summary	4
6. Findings	4
Low Severity	5
[L-01] Just-in-time mint of a child Folio captures the parent Folio's undistributed fee-recipient payout	5
[L-02] deprecateFolio leaves the TVL fee accruing forever while the documented wind-down destroys every off switch	7
[L-03] Nonce-unbound `endRebalance()` lets a stale authorized call silently terminate a later rebalance	8
[L-04] FolioLens omits balances held by an active trusted filler	10
[L-05] Redeeming the entire supply permanently disables future minting	11
Info Severity	13
[I-01] Minting into an emptied basket issues unbacked shares, enabling a near-total drain	13
[I-02] Redundant array copy in FolioLens.getAllBids can be replaced by in-place truncation	15
[I-03] `setName()` allows rebranding the Folio token but no equivalent exists for the symbol	17
[I-04] getAuctionPrice returns a stale nonzero price for a token whose executable size is permanently zero	17
[I-05] Trusted-fill double rounding rejects exact full-lot orders and can nearly double an enforced price	18
[I-06] Shared auction clock for newly available reverse pairs enables loss of Folio backing	19
[I-07] DAO fee-registry changes retroactively reprice unpoked TVL-fee periods	21
[I-08] Fixed-price auctions are forced to use maxAuctionLength	22



1. About Pashov Audit Group

Pashov Audit Group consists of 100+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 500 security audits completed - uncovering and helping patch thousands of vulnerabilities - the group strives to create the most robust and pleasant audit experience possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Telegram [@pashovkrum](#).

2. Disclaimer

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue has been reviewed; the outcome is recorded in the finding's Resolution section.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Critical

Loss or compromise of critical assets, or protocol-wide impact.

High

Significant material loss of assets, or serious harm to a group of users.

Medium

Moderate material loss, or moderate harm, often under conditional assumptions.

Low

Minor impact, limited scope, or requires unlikely conditions to trigger.

Info

No direct security impact - best-practice, code-quality or hardening suggestion.

IMPACT

High - leads to a significant material loss of assets in the protocol or significantly harms a group of users

Medium - leads to a moderate material loss of assets in the protocol or moderately harms a group of users

Low - leads to a minor material loss of assets in the protocol or harms a small group of users

LIKELIHOOD

High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost

Medium - only a conditionally incentivized attack vector, but still relatively likely



Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. Methodology

A large team of security researchers hunt vulnerabilities on the scope provided during a time-boxed review.

Every issue submission is deduplicated and validated by a judging team. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

5. Executive Summary

A time-boxed security review of Reserve was conducted by the Pashov Audit Group team. This report documents 13 issues across the reviewed codebase, reported by 0xBeastBoy, 0x133, alix2, BengalCatBalu, bibawen, Farman, IvanFitro, jesjupiter, K42, merlinboii, unforgiven, WarlordSam.

SCOPE

<https://github.com/reserve-protocol/reserve-index-dtf>

Everything under "contracts/"

REVIEW COMMIT HASH

[18706fb455b8e6b91250deba795eb791243f6827](https://github.com/reserve-protocol/reserve-index-dtf/commit/18706fb455b8e6b91250deba795eb791243f6827)

6. Findings

ID	TITLE	SEVERITY	STATUS
L-01	Just-in-time mint of a child Folio captures the parent Folio's undistributed fee-recipient payout	Low	Acknowledged
L-02	deprecateFolio leaves the TVL fee accruing forever while the documented wind-down destroys every off switch	Low	Fixed
L-03	Nonce-unbound `endRebalance()` lets a stale authorized call silently terminate a later rebalance	Low	Fixed
L-04	FolioLens omits balances held by an active trusted filler	Low	Fixed
L-05	Redeeming the entire supply permanently disables future minting	Low	Acknowledged
I-01	Minting into an emptied basket issues unbacked shares, enabling a near-total drain	Info	Acknowledged
I-02	Redundant array copy in FolioLens.getAllBids can be replaced by in-place truncation	Info	Acknowledged



I-03	`setName()` allows rebranding the Folio token but no equivalent exists for the symbol	Info	Acknowledged
I-04	getAuctionPrice returns a stale nonzero price for a token whose executable size is permanently zero	Info	Acknowledged
I-05	Trusted-fill double rounding rejects exact full-lot orders and can nearly double an enforced price	Info	Acknowledged
I-06	Shared auction clock for newly available reverse pairs enables loss of Folio backing	Info	Acknowledged
I-07	DAO fee-registry changes retroactively reprice unpoked TVL-fee periods	Info	Acknowledged
I-08	Fixed-price auctions are forced to use maxAuctionLength	Info	Acknowledged

Low Severity

[L-01] Just-in-time mint of a child Folio captures the parent Folio's undistributed fee-recipient payout

Low

Acknowledged

SUMMARY

A Folio is a plain ERC20, so one Folio can be a basket asset of another, and any address except the Folio itself can be a fee recipient (the only identity restriction is `recipient ≠ address(this)`, alongside ascending-order and nonzero-portion checks in `FolioLib.sol#L56-L64`). "Folio-of-Folios" fee routing is therefore explicitly allowed: parent Folio A pays its non-DAO fees to child Folio B, while B lists A as a basket asset.

The two sides price A inconsistently. Parent A counts its accrued-but-undistributed fee shares inside its own supply, so A's per-share value already reflects the fees A owes B. But B prices its holding of A at the raw on-chain balance, which does not yet include those pending shares. B's mint and redeem both route through `_toAssets()`, which reads each basket amount via `_balanceOfToken()`:

```
function _balanceOfToken(ERC20 token) internal view returns (uint256 amount) {  
    amount = token.balanceOf(address(this));  
}
```

`distributeFees()` is permissionless and is what mints A's pending recipient shares into B (`_mint(recipients[i].recipient, shares)`). A roleless actor mints a large fraction of B while B is priced without A's pending fees, then calls `A.distributeFees()` so those shares land in B, then redeems B and A. The freshly minted B shares claim a pro-rata slice of the just-materialized fee shares the actor never funded.



The root is the asymmetry: A prices the owed fees into its supply, B does not price them into its claim on A, so the value is capturable in the window before A distributes. This is distinct from the M-22 self-fee snapshot issue, which is a single-Folio problem; here parent A owes recipient-specific fee shares to child B and B misprices its held A. The two require different fixes.

IMPACT

An untrusted actor with no role on either Folio extracts value belonging to B's incumbent fee-recipient holders and repays its full principal in the same transaction, so the extracted amount is pure profit.

On a one-year TVL-fee accrual, the atomic round trip returns the entire 100,000.000000 USDC principal and nets 8,965.412212 USDC. The pending fee shares flowing into B on distribution are 11,111.073986347214360450 A shares, while B's listed A balance before distribution priced only 9,000 A shares. That gap is what the minter monetizes.

The loss is not capped to one fee event and survives realistic fee configurations:

Scenario	Attacker profit (USDC)	Note
One-year TVL accrual, principal repaid	8,965.412212	full principal returned
Default DAO cut + one-year accrual	5,887.280132	DAO one-third cut plus fee floor restored
B made an immutable fee recipient	8,965.412212	removal reverts Folio__ImmutableFeeRecipientRemoved
Single third-party parent mint at 5% fee	4,391.904656	4,970 A shares owed to B, captured same block
100 rounds honest parent churn	447,086.056556	incumbent loss 447,252.915790 of a 508,000.000000 claim (88.00%)
100 rounds churn + default DAO cut	299,411.242105	incumbent loss 299,968.421053 of a 344,333.333333 claim (86.95%)

Limitations: It requires the nested topology (B holds A and is a fee recipient of A). It cannot profit once the parent is already settled: the distribute-first control nets -85.868672 USDC, the ordinary 3 bps minimum mint fee friction. It mints no new shares and breaks no conservation; the captured value comes entirely from B's incumbent holders.

EXPLOIT PATH

1. Set up (or find) the permitted topology: parent A pays non-DAO fees to child B, and B lists A as a basket asset.
2. While B prices its held A at raw `balanceOf` (omitting A's pending recipient shares), mint a large fraction of B.
3. Call the permissionless `A.distributeFees()`, materializing A's pending recipient shares into B.
4. Redeem B, then A. The freshly minted B shares claim a pro-rata slice of the just-materialized fee shares, funded entirely by B's incumbent holders. Principal is repaid in the same atomic transaction.

PREREQUISITES

Requires the nested topology: B must hold A as a basket asset and be a fee recipient of A. No role is needed on either Folio. A pending receivable is the default state of the parent between distribution cycles (from any TVL fee accruing over time or any parent mint at a nonzero mint fee), and the actor triggers `distributeFees()` itself, so there is no honest transaction to front-run.

RECOMMENDATION

Make a Folio's holding of another Folio reflect that Folio's up-to-date entitlement before pricing any mint or redeem. Two options:

1. Settle the parent before the child prices it. When a basket asset is itself a Folio that lists the holder as a fee recipient, materialize the parent's pending recipient shares in the child's sync/pricing path so `A.balanceOf(B)` already includes B's receivable when `_toAssets()` runs:

```
for (each basket asset t that is a Folio) {
  IFolio(t).distributeFees();
}
```

2. Or price a held Folio share at its full redeemable value, including this holder's pro-rata share of that Folio's pending fee shares, rather than at raw `balanceOf`.

Apply either fix symmetrically to `mint` and `redeem`, so a redemption landing while the receivable is outstanding does not silently forfeit the redeeming holder's slice to the remaining holders. The fix should keep the distribute-first round trip unprofitable, keep an immutable fee recipient protected, and leave ordinary non-nested mint and redeem unaffected.

[L-02] deprecateFolio leaves the TVL fee accruing forever while the documented wind-down destroys every off switch

LowFixed

SUMMARY

`deprecateFolio` only flips a flag and emits an event. No fee code path reads `isDeprecated`, so TVL fee inflation keeps accruing at the full pre-deprecation rate even though mint, rebalancing, and every auction entry point are already disabled.

```
function deprecateFolio() external nonReentrant onlyRole(DEFAULT_ADMIN_ROLE) {
  isDeprecated = true;
  emit FolioDeprecated();
}
```

The only lever that stops accrual is `setTVLFee`, which is restricted to `DEFAULT_ADMIN_ROLE`. The shipped wind-down procedure in `script/deprecation/README.md` revokes `DEFAULT_ADMIN_ROLE` and renounces `FolioProxyAdmin` ownership without ever calling `setTVLFee(0)`. Once the admin member count reaches zero the role can never be re-granted, and the upgrade path is closed against `address(0)`.

The DAO cannot fix this from the registry side either. `computeFeeShares` uses `_tvlFee = max(feeFloor, tvlFee)` and the registry only controls the split, so zeroing every registry knob leaves the inflation unchanged and routes all of it to the now frozen fee recipient table.

IMPACT

Every fully deprecated Folio permanently dilutes its remaining holders at the pre-deprecation TVL fee rate, up to 10.00 percent per year at the maximum, compounding without bound, with no address able to reduce it.

The new shares mint to a fee recipient table that can never be changed, and the DAO receives none of it after zeroing its own knobs. The entire loss is a transfer from redemption-only holders to a frozen recipient. Any subsequent permissionless `poke` , `redeem` , or `distributeFees` mints new fee shares at the frozen rate on a vehicle that performs no management at all.

RECOMMENDATION

Terminate fee accrual atomically inside `deprecateFolio` : call `distributeFees()` , then set `tvfFee = 0` , before setting `isDeprecated = true` . Alternatively, have `_getFeeShares` return the current pending values unchanged whenever `isDeprecated` is set, so no further shares can be minted.

Add a regression test that deprecates a Folio at a nonzero `tvfFee` , advances one year, and asserts `totalSupply` and per-share redemption value are unchanged across the jump. Extend the deprecation fork suite to assert the same after all roles are revoked.

RESOLUTION - FIXED

<https://github.com/reserve-protocol/reserve-index-dtf/pull/218>

[L-03] Nonce-unbound `endRebalance()` lets a stale authorized call silently terminate a later rebalance

Low

Fixed

SUMMARY

Folio v6 drives a rebalance through a set of role-gated lifecycle operations, and every one is bound to the specific rebalance nonce it was authorized for, except `endRebalance()` . `startRebalance` requires the exact next nonce (`RebalancingLib.startRebalance:42-43`), and both `openAuction` and `openAuctionUnrestricted` flow through the nonce gate `rebalance.nonce = rebalanceNonce` inside `_openAuction` . `closeAuction` targets an exact monotonic auction id. `endRebalance` takes no argument and, after only a role check, unconditionally writes the current rebalance's `availableUntil` :

```
function endRebalance() external nonReentrant {
  _checkPrivileged();
  emit RebalanceEnded(rebalance.nonce);
  // do not revert, to prevent griefing
  rebalance.availableUntil = block.timestamp; // exclusive
}
```

Because the call carries no target, an `endRebalance` transaction formed and authorized while one rebalance was current is byte-identical to one formed for the next rebalance. If it executes after another manager has ended the first rebalance and started the next, it acts on whatever rebalance is current and terminates the fresh one instead of the intended one.

`REBALANCE_MANAGER` is expected to be a governance timelock, and a `Succeeded` governor proposal stays publicly executable, so a queued end action remains valid after the nonce has advanced. The codebase

invites exactly this bundling: `removeFromAllowlist` documents `endRebalance()` as a natural follow-on step, which is how a nonce-less end action ends up queued alongside another operation.

```
/// Remove tokens from the allowlist
/// @dev Does not impact ongoing rebalances. Consider calling endRebalance()
```

IMPACT

Once `availableUntil` is set to the current timestamp, the nonce gate in `_openAuction` rejects any new auction with `Folio__NotRebalancing`, so the fresh rebalance can no longer open auctions:

```
require(
  rebalance.nonce == rebalanceNonce &&
  block.timestamp >= rebalance.startedAt + auctionBuffer &&
  block.timestamp < rebalance.availableUntil,
  Folio__NotRebalancing()
);
```

Rebalance state only narrows within a nonce, so the sole recovery is a fresh `startRebalance`, which mints a new nonce and, for a timelocked `REBALANCE_MANAGER`, costs a full governance round-trip.

While the terminated rebalance is halted, the basket sits at whatever partial, off-target composition it had reached. If the interruption coincides with a market move, the Folio can be left holding an unintended composition until governance restarts, a downstream value risk to holders.

This moves, mints, and redeems no assets and does not bypass conservation. It does not stop mint or redeem, and it does not close an already-open auction. It also does not require the authorizer to act maliciously: the authorizer honestly intended to end the earlier rebalance, and the defect is that the approved calldata silently retargets the later one.

EXPLOIT PATH

1. A `REBALANCE_MANAGER` authorizes an `endRebalance()` action (for example bundled with `removeFromAllowlist`) while rebalance nonce N is current, but it is not executed immediately.
2. Another manager ends rebalance N and calls `startRebalance`, advancing to nonce N+1.
3. The still-valid, still-executable end action from step 1 is executed and lands on nonce N+1, setting its `availableUntil` to the current timestamp.
4. The nonce gate in `_openAuction` now rejects every new auction for N+1 with `Folio__NotRebalancing`.
5. Recovery requires a brand-new `startRebalance`, i.e. a full governance round-trip for a timelocked manager.

RECOMMENDATION

Give `endRebalance` an explicit target nonce and require it to match the current rebalance nonce:

```
function endRebalance(uint256 rebalanceNonce) external nonReentrant {
    _checkPrivileged();
    require(rebalance.nonce == rebalanceNonce, Folio__InvalidRebalanceNonce());
    emit RebalanceEnded(rebalance.nonce);
    rebalance.availableUntil = block.timestamp; // exclusive
}
```

This preserves the original anti-griefing intent. Ending a rebalance does not advance the nonce, so a second call for the same current nonce still matches and stays idempotently successful. Reverting on a wrong nonce does not reintroduce griefing, because the caller controls the argument and can only revert its own stale calldata.

Update every caller and the ABI, and update any optimistic-governor selector configuration that references the `endRebalance` selector, so pre-authorized end actions encode the nonce they intend to end.

Regression coverage a fix must keep passing: a second `endRebalance` for the same current nonce still succeeds and emits `RebalanceEnded`; an `endRebalance` carrying a stale nonce reverts `Folio__InvalidRebalanceNonce`; an ongoing auction is left untouched; and the sibling operations keep their existing nonce bindings.

RESOLUTION - FIXED

<https://github.com/reserve-protocol/reserve-index-dtf/pull/216>

[L-04] FolioLens omits balances held by an active trusted filler

Low

Fixed

LOCATION

FolioLens:getSpotWeights:24-40

SUMMARY

The spot-weight view should use the same economic asset balances as the Folio's own accounting. Assets temporarily held by an active trusted filler stay attributable to the Folio and are counted by `_balanceOfToken()`.

`getSpotWeights()` instead reads only the Folio's direct ERC-20 balances:

```
uint256 totalSupply = folio.totalSupply();
for (uint256 i = 0; i < tokens.length; i++) {
    address token = tokenParams[i].token;
    weights[i] = (D27 * IERC20(token).balanceOf(address(folio))) / totalSupply;
}
```

Whenever a trusted fill is active, the lens reports a composition that diverges from the Folio's share and auction accounting.



IMPACT

Off-chain consumers can receive incorrect token-per-share weights while an asynchronous trusted fill holds assets outside the Folio address. The affected token's weight is understated relative to its true economic balance, which can feed stale valuation or auction-planning decisions.

This is a view-only issue: it does not alter Folio accounting or move funds.

EXPLOIT PATH

1. A trusted fill is created and the filler contract holds part of the sell or acquired buy balance outside the Folio address.
2. `Folio._balanceOfToken()` includes that filler balance for auction and asset calculations.
3. An external consumer calls `FolioLens.getSpotWeights()`, which reads only `balanceOf(folio)`.
4. The returned weight for the affected token stays understated until the trusted fill closes.

PREREQUISITES

Trusted fills must be enabled and an active fill must currently hold a configured sell or buy token outside the Folio address.

RECOMMENDATION

Add a public view in Folio that exposes the trusted-fill-aware balance and use it from the lens:

```
function balanceOfAsset(IERC20 token) external view returns (uint256) {  
    return _balanceOfToken(token);  
}
```

Then replace the direct balance read:

```
uint256 bal = folio.balanceOfAsset(IERC20(token));  
weights[i] = (D27 * bal) / totalSupply;
```

RESOLUTION - FIXED

<https://github.com/reserve-protocol/reserve-index-dtf/pull/215>

[L-05] Redeeming the entire supply permanently disables future minting

Low

Acknowledged

LOCATION

Folio:_toAssets:682-696

SUMMARY

When the final holder redeems the entire effective share supply, the Folio drops to a zero `totalSupply()`. On the next mint, `_toAssets()` reads that zero supply and uses it directly as the division denominator:

```
uint256 _totalSupply = totalSupply();
(_assets, _amounts) = _totalAssets();
for (uint256 i; i < _assets.length; i++) {
    _amounts[i] = Math.mulDiv(shares, _amounts[i], _totalSupply, rounding);
}
```

With `_totalSupply == 0`, the `Math.mulDiv` reverts. Because minting is the only ordinary operation that could restore share supply, the reverting denominator leaves the Folio unable to recover through the normal interface.

The zero-supply state can also be reached by redeeming the final shares to the Folio itself, which retains the underlying basket assets while blocking both mint and redemption. Any residual unsolicited basket balances keep the asset list nonempty, so the Folio looks live but the zero denominator has bricked it.

IMPACT

A complete redemption can leave the Folio permanently unable to accept future mints, even when it is otherwise active and intended to take new deposits.

This is a protocol-level availability and lifecycle failure. It is most reachable for fee-free Folios, or Folios whose pending fee shares have been fully settled, since there are no residual fee shares to keep supply above zero.

EXPLOIT PATH

1. A live Folio has no effective pending fee shares and one holder owns the entire circulating supply.
2. The sole holder calls `redeem(S, holder, assets, minAmountsOut)` with `S` equal to the full effective `totalSupply`, burning supply to zero.
3. A later user calls `mint(S2, receiver, minSharesOut)`.
4. `mint()` invokes `_toAssets()`, whose `Math.mulDiv` divides by `_totalSupply == 0` and reverts, permanently preventing future minting.

RECOMMENDATION

Reject any redemption that would leave an active Folio with zero effective supply:

```
uint256 supply = totalSupply();
require(shares < supply, Folio__ZeroSupplyNotAllowed());
(_assets, _amounts) = _toAssets(shares, Math.Rounding.Floor);
```

Alternatively, add a dedicated zero-supply bootstrap path with explicit governance or depositor authorization, and guard the denominator before calling `Math.mulDiv`:

```
if (_totalSupply == 0) {
    revert Folio__ZeroSupplyRequiresBootstrap();
}
```

Info Severity

[I-01] Minting into an emptied basket issues unbacked shares, enabling a near-total drain

Info

Acknowledged

SUMMARY

`initialize()` requires a non-empty basket and only adds assets with a nonzero balance, but nothing preserves that invariant afterward. `mint()` prices the caller's deposit against the current basket through `_toAssets()`, which reads `_totalAssets()`, and `_totalAssets()` is simply `basket.values()` plus each member's balance.

When the basket is empty, `_toAssets()` returns empty arrays, so the transfer loop pulls nothing while `_mint()` still issues the full requested amount:

```
(_assets, _amounts) = _toAssets(shares, Math.Rounding.Ceil);
uint256 assetLength = _assets.length;
for (uint256 i; i < assetLength; i++) {
    if (_amounts[i] != 0) {
        SafeERC20.safeTransferFrom(IERC20(_assets[i]), msg.sender, address(this), _amounts[i]);
    }
}
_mint(receiver, sharesOut);
```

On an empty basket `assetLength` is zero, so the loop runs zero times and pulls nothing, yet `_mint()` still issues the full post-fee `sharesOut`. `mint()` is external and roleless, guarded only by `nonReentrant notDeprecated sync`. A caller with no role can mint an unbacked, supply-dominating share balance for zero cost.

The empty state is reachable by an untrusted actor around ordinary basket management. Manual `removeFromBasket()` has no last-member guard, and two settlement paths auto-remove a fully-sold token: a full direct `bid()` removes its now-zero sell token, and closing a fully-settled trusted fill removes its zero-balance sell token. By fully trading the last remaining basket token into a token that an already-ready honest removal proposal targets, then executing that ready removal through the public Governor entrypoint, a roleless actor can drive the basket to empty in a single transaction and then mint.

This is distinct from the known items M-52 and M-58. M-52's post-transfer per-asset check iterates over basket members, so on an empty basket it runs zero times and never fires. M-58 reverts only when `totalSupply()` is zero; here `totalSupply()` is nonzero (initial plus outstanding fee shares) while the basket is empty, so no denominator is zero and shares are issued for free instead. Closing this requires a different change from either fix.

IMPACT

A roleless attacker can mint a supply-dominating amount of shares for free, then extract value along two independently demonstrated paths.

Immediate, irreversible external loss via standing secondary liquidity. Selling the free shares into a constant-product pool seeded at the Folio's honest two-stablecoin NAV extracts `1999999999` of `2000000000` raw USDC units (about 99.99999995 percent), leaving one raw unit, while the pool invariant



stays nondecreasing. This loss lands on external liquidity providers and no later administrative repair of the Folio can undo it.

Near-total capture of the Folio's backing on honest recovery. In the USDC/DAI variant, after the attacker's fair auction fill the Folio holds about 20,000 DAI off-basket (`20000000000002289000000` at 18 decimals). Because the free shares dominate supply, once governance honestly restores the token with `addToBasket()`, the attacker redeems and nets `18994516271509672062943` stablecoin units, about 94.97 percent of the Folio's entire holding at recovery. The mint requests `999999999998985559977927105837433` shares, the remaining capacity below the documented `1e36` supply ceiling.

Secondary denial of service. An oversized empty-basket mint drives effective supply high enough that the fee-share math overflows at the next daily UTC fee checkpoint, after which `totalSupply()`, `poke()`, and `redeem()` all revert. An honest basket re-add does not restore them without an implementation fix.

Limitations: the free-mint transaction performs no direct token theft (it calls `transferFrom` zero times), and the attacker cannot remove the last basket token alone since `removeFromBasket()` is admin-gated. The empty state is reached only by front-running an already-ready honest removal or driving an auto-removal through a fair bid/fill, and monetization needs either an honest `addToBasket()` recovery or standing external secondary liquidity.

EXPLOIT PATH

1. Wait for an in-progress rebalance whose launcher-restricted window has elapsed, so `openAuctionUnrestricted` is publicly callable, and for an already-ready honest `removeFromBasket` proposal targeting the traded-into token.
2. Fully trade the last remaining basket token into that targeted token via a fair roughly 1:1 fill at the Dutch-auction midpoint, so the sold-out token auto-removes.
3. Execute the ready removal through the public Governor `execute()`, driving the basket to empty.
4. Call `mint()` against the empty basket to receive supply-dominating shares while transferring nothing.
5. Monetize either by selling the free shares into standing secondary liquidity, or by waiting for an honest `addToBasket()` recovery and redeeming.

PREREQUISITES

No Folio role is required. The attacker supplies only execution timing. The attack needs an in-progress rebalance with an elapsed launcher-restricted window, an already-ready honest `removeFromBasket` proposal for the traded-into token, and a fair roughly 1:1 fill at the auction midpoint.

RECOMMENDATION

Reject unbacked issuance in `mint()`. After computing `_toAssets`, require a non-empty basket and that at least one asset carries a strictly positive required amount, so a caller cannot receive shares without transferring backing:

```
(_assets, _amounts) = _toAssets(shares, Math.Rounding.Ceil);
require(_assets.length != 0, Folio__EmptyBasket());
bool chargedSomething;
uint256 assetLength = _assets.length;
for (uint256 i; i < assetLength; i++) {
    if (_amounts[i] != 0) {
        chargedSomething = true;
        SafeERC20.safeTransferFrom(IERC20(_assets[i]), msg.sender, address(this), _amounts[i]);
    }
}
require(chargedSomething, Folio__ZeroBackingMint());
```

Preserve the non-empty invariant that `initialize()` establishes. Prevent `removeFromBasket()` and the auto-removal paths (`bid()` and `_closeTrustedFill()`) from removing the final basket member while `totalSupply()` is nonzero; require an explicit replacement asset before the last member can leave.

Optionally, bound share issuance so a single mint cannot alone push effective supply past the documented `1e36` ceiling, closing the fee-checkpoint overflow denial of service independently of the backing fix.

Regression coverage a fix must keep passing: a mint against an empty basket reverts and issues zero shares; a mint against a one-unit residual basket still requires a positive, correctly-scaled deposit; removing or auto-removing the last basket member while shares are outstanding reverts (or requires a replacement) for the manual, direct-bid, and trusted-fill paths; no roleless sequence can reach a supply above the ceiling; and honest single-asset and two-asset lifecycles continue to succeed.

[I-02] Redundant array copy in `FolioLens.getAllBids` can be replaced by in-place truncation

InfoAcknowledged

SUMMARY

`getAllBids` over-allocates a result array of size `len * len`, populates the first `count` entries, then allocates a second array and copies those entries across to drop the unused tail.

```
SingleBid[] memory allBids = new SingleBid[](len * len);
// ... fills allBids[count] for valid pairs, count++ ...
bids = new SingleBid[](count);
for (uint256 i = 0; i < count; i++) {
    bids[i] = allBids[i];
}
```

Since the array is only ever shrunk (`count ≤ len * len`) and indices `0..count-1` are already populated, the second allocation and copy loop are unnecessary. The length word of `allBids` can simply be truncated in place to hide the unused tail.

IMPACT

`getAllBids` is a read-only helper returning data to off-chain callers; no funds or on-chain state depend on it. Removing the extra allocation and the `0(count)` copy loop marginally reduces gas and simplifies the code.

**RECOMMENDATION**

Replace the second allocation and copy loop with an in-place length truncation:

```
assembly {  
  mstore(allBids, count)  
}  
bids = allBids;
```

[I-03] `setName()` allows rebranding the Folio token but no equivalent exists for the symbol

Info

Acknowledged

SUMMARY

The admin can change a Folio's ERC20 name at any time via `setName()`, which writes directly into the ERC20 storage slot. There is no matching `setSymbol()`, so the symbol is locked at initialization forever.

```
function _setName(string calldata _newName) internal {
    ERC20Storage storage $;
    assembly {
        $.slot := ERC20_STORAGE_LOCATION
    }
    $_name = _newName;
    emit NameSet(_newName);
}
```

IMPACT

A Folio that rebrands (common for index products whose mandate or branding evolves) can update its display name but keeps the old ticker. Wallets, explorers and integrations then show a mismatched pair of new name and stale symbol, which is confusing for holders and undercuts the purpose of `setName`.

Conversely, if metadata was meant to be immutable, `setName` already breaks that expectation for the name alone.

RECOMMENDATION

Add the symmetric setter:

```
function setSymbol(string calldata _newSymbol) external onlyRole(DEFAULT_ADMIN_ROLE) {
    ERC20Storage storage $;
    assembly {
        $.slot := ERC20_STORAGE_LOCATION
    }
    $_symbol = _newSymbol;
    emit SymbolSet(_newSymbol);
}
```

[I-04] `getAuctionPrice` returns a stale nonzero price for a token whose executable size is permanently zero

Info

Acknowledged

SUMMARY

`getAuctionPrice` reports a price range using a single validity check, and that check is the one piece of state token removal never clears. It reads `auctions[auctionId].prices[token]` and requires only `range.low  $\neq$  0`, never consulting `rebalance.details[token].inRebalance`, basket membership, the auction time window, or the `rebalanceNonce`.

```
function getAuctionPrice(uint256 auctionId, address token) external view returns (PriceRange
memory range) {
    range = auctions[auctionId].prices[token];
    require(range.low != 0, Folio__InvalidAsset());
}
```

Reaching the stale state is permissionless. When a sell leg is bought down to a zero balance (the normal terminal state of a weight-zero sell leg), `Folio.bid` calls `_removeFromBasket(sellToken)`, which does `basket.remove(token)` and `delete rebalance.details[token]` but never deletes `auctions[id].prices[token]`.

After such a sellout, `getAuctionPrice` still returns the original nonzero range on an auction whose `rebalanceNonce` still matches and whose time window still contains `block.timestamp`, so the quote carries no staleness signal. Every other reader disagrees in the same block: `getBid` reverts `Folio__AuctionNotOngoing` because its `_price` path additionally requires `rebalance.details[token].inRebalance`, and `FolioLens.getAllBids` returns zero biddable pairs.

IMPACT

An integrator or keeper reading `getAuctionPrice` sees a live nonzero price range for a leg whose executable size is permanently zero, contradicting `getBid` and `getAllBids`. Any bid built from that quote reverts.

The blast radius is small: the function is view, moves no funds, and has no in-repo consumer. The harm is limited to a wrong price handed to an off-chain or cross-contract consumer plus a guaranteed-revert bid built on it.

RECOMMENDATION

Give `getAuctionPrice` the same gate the other auction readers use, extending its require to also check `rebalance.details[token].inRebalance && auctions[auctionId].rebalanceNonce == rebalance.nonce`. Alternatively, have `_removeFromBasket` also delete the stale `auctions[id].prices[token]` entry.

Add a regression test asserting that after a permissionless sellout removes a token, `getAuctionPrice` for that token reverts rather than returning the original range.

[I-05] Trusted-fill double rounding rejects exact full-lot orders and can nearly double an enforced price

InfoAcknowledged

SUMMARY

`RebalancingLib.getBid()` returns a sell amount `S`, a buy amount `B = ceil(S * A / D27)`, and the D27 price `A` (`contracts/utils/RebalancingLib.sol:277-345`). `Folio.createTrustedFill()` discards `A` and initializes the filler with only `(S, B)` (`contracts/Folio.sol:860-891`).

The pinned `CowSwapFiller` then recomputes its own price `P = ceil(B * D27 / S)` and accepts an order `(s, b)` only if `floor(b * D27 / s) ≥ P` (`CowSwapFiller.sol:56-76,79-100` at commit `303e0a56948b602365a1ed0024ed473f683aeb0f`, pinned by `package.json:11`).

This is a double-rounding mismatch: for an order using the exact Folio quote `(S, B)`, whenever `(B * D27) % S ≠ 0` the validation value is exactly `P - 1`, so the filler reverts. Partial-fill or oversized nominal encodings do not restore the quoted ratio either, since every accepted order satisfies `b / s ≥ P / D27`.

$> B / S$; CoW settlement enforces the signed limit, so a full sale of S requires at least $\text{ceil}(S * P / D27) > B$.

Concrete boundary example: a 27-decimal sell token holds surplus $S0 = 999,999,999 * D27$, the buy token has 0 decimals, and $A = 1$. The initial quote is $(B0, P0) = (999,999,999, 1)$. One donated sell-token quantum makes $S = S0 + 1$; the Folio quotes $B = 1,000,000,000$ but the filler stores $P = 2$. The exact order evaluates to 1 and reverts, and a fully accepted sale requires about $1,999,999,999$ buy units, roughly a 100% premium over the Folio quote. The existing integration test happens to pick an integral ratio, so it never triggers the failure (`test/Folio.t.sol:2310-2340`).

IMPACT

The exact full-lot order the Folio produces is rejected for every nonintegral reconstructed ratio.

For coarse supported denominations, one extra D27 price quantum is economically material, and an untrusted one-quantum sell-token donation can nearly double the consideration required to fill the full lot.

With direct bids disabled (`CHANGELOG.md:18`), this can make the only configured route infeasible at the approved rate, delaying rebalancing and leaving holders exposed to composition and market risk.

RECOMMENDATION

Propagate the quoted D27 price A from `getBid()` through `createTrustedFill()` into the filler, and validate orders against that single price rather than recomputing $P = \text{ceil}(B * D27 / S)$ inside the filler. Using one consistent price and rounding direction end to end removes the double-rounding gap so that the exact quoted (S, B) order always validates.

[I-06] Shared auction clock for newly available reverse pairs enables loss of Folio backing

Info

Acknowledged

SUMMARY

Auction availability is recalculated from live balances, but every directed pair shares the same auction clock. If a balance change makes the $B \rightarrow A$ direction available late, that direction inherits the already-elapsed price curve instead of getting fresh price discovery. Past the curve's break-even point, the quote transfers value from the Folio to the bidder.

`RebalancingLib._price()` computes the D27-scaled start and end prices for a direction and decays between them over the auction's elapsed time:

```
S_AB = ceil(A.high * D27 / B.low)
E_AB = ceil(A.low * D27 / B.high)
P_AB(t) = max(E_AB, ceil(S_AB * exp(-k_AB * t) / D18))
```

Swapping A and B produces S_{BA} , E_{BA} , and P_{BA} , but both directions use the same `auction.startTime`, `auction.endTime`, and elapsed time. Dropping the fixed-point scaling and letting $u = t / T$, the two same-time prices multiply to $p_{AB}(u) * p_{BA}(u) = R^{(1-2u)}$, where R is the start/end price ratio. For $u < 0.5$ the round trip forces the bidder to add value; at $u = 0.5$ it breaks even; for $u > 0.5$ the bidder keeps value taken from the Folio. Rounding may shift the exact break-even point slightly, but the reverse cycle becomes profitable once the two implemented prices multiply to less than 1 after removing their `D27` scaling.

For exact-transfer ERC-20s this cannot be exploited, because the balance limits (not a price-product check) block the reverse direction. `RebalancingLib.getBid()` enforces $x \leq \text{balanceA} - H_A$ and $y \leq L_B - \text{balanceB}$, so after a fill $\text{balanceA}' \geq H_A \geq L_A$ (A never becomes buyable) and $\text{balanceB}' \leq L_B \leq H_B$ (B never becomes sellable). A fee checkpoint changes effective supply but moves every token target monotonically in the same direction, so it also cannot cross both opposite targets.

Rebasing and share-accounting tokens break this protection. The `README` treats upward- and downward-rebasing ERC-20s as supported, yet their `balanceOf` can cross a target with no transfer, because they store internal shares and derive balances from a changing conversion rate (Aave aTokens, stETH-style assets); conversion rounding can also decide which side of a target the balance lands on. `getBid()` reads live balances every call and does not remember whether a token started with sell surplus or buy deficit, and `_price()` gives the newly available direction no fresh curve.

IMPACT

A late reverse bid sells Folio assets at the unfavorable end of a curve the new direction never traversed. The bidder receives more market value than it pays and the Folio loses the difference, all while both token prices stay inside their approved ranges.

Take token A worth 1 UoA with an approved band of $[0.95, 1.05]$ and token B worth 4 UoA with band $[3.80, 4.20]$ (ordinary 5% bands). A fresh $B \rightarrow A$ curve would start at $4.20 / 0.95 = 4.421$ A per B, but inheriting the old auction's endpoint it immediately uses $3.80 / 1.05 = 3.619$ A per B:

```
bidder receives 1,000 B worth 4,000 UoA
bidder pays 3,619.048 A worth 3,619.048 UoA
Folio loss = 380.952 UoA, or 9.52% of the output value
```

The share supply does not change, so the loss directly reduces the basket value backing every Folio share.

The distinct balance-change paths that expose the stale reverse direction:

- **Downward rebase only:** A is at target, B is already above its sell target; a downward rebase pushes A below its buy target, making $B \rightarrow A$ available late.
- **Upward rebase only:** A is already below its buy target, B is at target; an upward rebase pushes B above its sell target, making $B \rightarrow A$ available late.
- **Opposing rebases after a fill:** an exact forward fill leaves A at its high target and B at its low target; a downward rebase of A plus an upward rebase of B crosses both targets and reverses the pair in the same auction.

This is not covered by existing known issues: M-28 concerns rebasing during settlement and corrupted `auction.traded`, whereas here rebases happen between completed bids and the counters stay accurate. M-67 concerns positive-rebase yield being classified as sellable surplus; even if that yield is excluded, an already-surplus B plus a later downward rebase of A still exposes $B \rightarrow A$ at the elapsed price.



EXPLOIT PATH

1. A rebalance auction on pair A/B is running and passes the curve midpoint.
2. A supported rebasing or share-accounting token undergoes a balance change (or an exact forward fill plus opposing rebases) that makes the reverse $B \rightarrow A$ direction available.
3. A searcher calls `Folio.bid()` for $B \rightarrow A$; `getBid()` reads the new live balances and allows it, while `_price()` still quotes the decayed endpoint price.
4. The searcher receives more market value in A than the B they pay, before the auction launcher can close or replace the auction.

Note: the `README` assigns the `AUCTION_LAUNCHER` responsibility for ending a dangerous rebalance, but this response is not enforced onchain by `Folio.bid()`, so a searcher can bid ahead of the launcher's closing transaction.

PREREQUISITES

The affected pair must involve a supported rebasing or share-accounting token whose displayed balance can cross a target without a transfer. The reverse direction must have sufficient capacity, and the bid must execute after the break-even point and before the launcher closes or replaces the auction. Gradual interest accrual gives the launcher more time to react, lowering (but not eliminating) the likelihood.

RECOMMENDATION

Lock the executable directions per auction. When an auction opens, snapshot the directed pairs that currently have capacity and permit only those directions for the life of the auction. If later balance changes enable a new direction, require opening a new auction with a fresh price curve rather than serving the new direction off the elapsed curve.

[I-07] DAO fee-registry changes retroactively reprice unpoked TVL-fee periods

Info

Acknowledged

LOCATION

`FolioDAOFeeRegistry:setDefaultFeeFloor/setDefaultFeeNumerator:65-85; Folio:_getFeeShares:840-884`

SUMMARY

Each elapsed TVL-fee period is meant to use the DAO fee numerator and fee floor active during that period, so registry policy changes should only affect future accrual.

Instead, each Folio stores only `lastPoke` and reads registry parameters at the moment the next `_getFeeShares()` runs. Because the default registry setters do not synchronize Folios, a policy change is applied to the entire stale interval:

```
if (_accountedUntil > lastPoke) {
  (...) = FolioLib.computeFeeShares(
    params,
    daoFeeRegistry
  );
}
```



`computeFeeShares()` queries the registry at calculation time, so the Folio cannot separate time before and after the registry update, producing a persistent discrepancy in fee shares and dilution.

IMPACT

DAO fees, recipient fees, self-fee allocation, and the resulting share dilution can differ from the policy that was actually active during the elapsed period.

A later increase overcharges users for past time, while a decrease permanently undercharges the DAO or other beneficiaries.

EXPLOIT PATH

1. At t_0 , a Folio is synchronized while the registry returns numerator/floor P_0 , and `lastPoke` is set to t_0 .
2. The Folio stays inactive while several daily TVL-fee boundaries pass.
3. At t_1 , the registry owner changes the default numerator or fee floor to P_1 without synchronizing this Folio.
4. At t_2 , a user calls `poke()`, `mint()`, `redeem()`, or `distributeFees()`, and `_getFeeShares()` prices the entire t_0 -to- t_2 interval using P_1 , including the period before t_1 .

PREREQUISITES

A Folio must have an unaccounted TVL-fee interval spanning a registry default-parameter update, and the registry owner must change the default numerator or floor before the Folio's next synchronization.

RECOMMENDATION

Synchronize all affected Folios before changing global defaults:

```
function setDefaultFeeFloor(uint256 newFloor) external onlyOwner {
    _pokeRegisteredFolios();
    defaultFeeFloor = newFloor;
}
```

A more scalable fix is to maintain timestamped registry parameters and pass the relevant historical values into fee calculation:

```
struct FeeConfig {
    uint256 numerator;
    uint256 floor;
    uint256 effectiveAt;
}
FeeConfig[] public feeHistory;
```

[I-08] Fixed-price auctions are forced to use `maxAuctionLength`

Info

Acknowledged

LOCATION

Folio:openAuction:660-672



SUMMARY

Fixed-price auctions (using `PriceControl.NONE`) are intended to accept any duration within the minimum and configured maximum limits. Removing price decay does not require the auction to last exactly the global maximum.

Instead, the `NONE` branch rejects every duration other than `maxAuctionLength`. When `maxAuctionLength` is one week, a launcher requesting a 120-second auction hits a `120 == 604800` check that fails, so the call reverts and no auction is created.

This is a functionality and documentation mismatch: it narrows the supported auction lifecycle but does not transfer funds or freeze assets.

IMPACT

Auction launchers cannot choose a shorter duration for fixed-price auctions. A configuration with a long `maxAuctionLength` forces unnecessarily long auctions or reliance on a privileged account to close them early, which can delay operational rebalances.

EXPLOIT PATH

1. Configure `maxAuctionLength` to one week and start a rebalance with `PriceControl.NONE`.
2. The launcher calls `openAuction` with `auctionLength = 120` seconds.
3. The `NONE` branch requires `120 == 604800`, so the call reverts and no auction is created.
4. The launcher must open a one-week auction or have a privileged account close it early.

PREREQUISITES

The rebalance must use `PriceControl.NONE`, and `maxAuctionLength` must be greater than the shorter duration the launcher operationally needs.

RECOMMENDATION

Replace the exact-equality check with the documented bounded-range check:

```
require(
  auctionLength >= MIN_AUCTION_LENGTH &&
  auctionLength <= maxAuctionLength,
  Folio__InvalidAuctionLength()
);
```

This permits shorter fixed-price auctions while preserving the configured minimum and maximum duration limits.